# Numerical Analysis and Optimization of BJT Transistor Bias Circuit Using Root-Finding Methods

Complex Engineering Activity Report

Course Title: Applied Numerical Methods
Course Code: MT-471
TE Batch: 2023, Fall Semester 2025

Submitted By:
Tooba Fayyaz (SE-23071)
Laiba Majeed (SE-23302)

Instructor: Engr. Faraz

# Contents


**1 Introduction and Problem Background** **2**

**2 Mathematical Formulation** **2**

**3 Algorithm Design and Numerical Methods** **2**
- 3.1 Bisection Method . . . . . 2
- 3.2 Newton-Raphson Method . . . . . 3
- 3.3 Secant Method . . . . . 3

**4 Python Implementation** **3**
- 4.1 Code Description . . . . . 3
- 4.2 Complete Python Code . . . . . 3

**5 Convergence Testing and Performance Evaluation** **6**
- 5.1 Computed Results . . . . . 6
- 5.2 Convergence Curves . . . . . 6
- 5.3 Iteration Comparison . . . . . 6

**6 Optimization Insights and Real-World Relevance** **7**

**7 Discussion of Results** **7**

**8 Conclusion** **7**

# 1 Introduction and Problem Background

Accurate modeling of semiconductor devices, particularly the Bipolar Junction Transistor (BJT), is fundamental to analog circuit design. Determining the transistor operating point (Q-point) is crucial for ensuring proper linearity, gain, and overall stability of circuits. Unlike purely resistive networks, BJT currents and voltages are governed by a nonlinear exponential relationship, making direct analytical solutions for base-emitter voltage ($V_{BE}$) impossible. Consequently, numerical root-finding techniques such as Bisection, Newton-Raphson, and Secant methods become essential for determining $V_{BE}$ for a given base current ($I_B$). This activity aligns with CLO-3 and addresses complex problem-solving attributes (CPA-1, CPA-2, CPA-3), requiring abstract thinking, interaction of multiple technical issues, and extension beyond previous experience.

# 2 Mathematical Formulation

The BJT base-emitter junction behaves as a diode, obeying the Shockley diode equation:

$$I_B = I_S \left( e^{\frac{V_{BE}}{\eta V_T}} - 1 \right) \tag{1}$$

where $I_S$ is the saturation current, $\eta$ is the ideality factor, $V_T$ is the thermal voltage, and $I_B$ is the target base current. For this study, the parameters are:

$$I_S = 10^{-12} \text{ A}, \quad \eta = 1, \quad V_T = 0.025 \text{ V}, \quad I_B = 4.545 \times 10^{-5} \text{ A}$$

The nonlinear equation can be rewritten as a root-finding problem:

$$f(V_{BE}) = I_S \left( e^{\frac{V_{BE}}{\eta V_T}} - 1 \right) - I_B = 0 \tag{2}$$

This represents the operating point of the transistor, which we aim to compute using numerical methods.

# 3 Algorithm Design and Numerical Methods

Three classical methods were implemented in Python to solve for $V_{BE}$: Bisection, Newton-Raphson, and Secant. Each method has distinct characteristics in terms of convergence, computational cost, and reliability.

## 3.1 Bisection Method

The Bisection Method uses the Intermediate Value Theorem: if $f(a)$ and $f(b)$ have opposite signs, a root exists in $[a, b]$. The interval is halved iteratively until the root is approximated within a specified tolerance. While guaranteed to converge, it is relatively slow.

### 3.2 Newton-Raphson Method

Newton-Raphson uses tangent line approximations for iterative root-finding:

$$V_{BE}^{(n+1)} = V_{BE}^{(n)} - \frac{f(V_{BE}^{(n)})}{f'(V_{BE}^{(n)})} \tag{3}$$

requiring the derivative:

$$f'(V_{BE}) = \frac{I_S}{\eta V_T} e^{\frac{V_{BE}}{\eta V_T}} \tag{4}$$

It converges quadratically, making it the fastest among the three, but requires a good initial guess.

### 3.3 Secant Method

The Secant Method approximates the derivative numerically using two previous points, avoiding explicit differentiation:

$$V_{BE}^{(n+1)} = V_{BE}^{(n)} - f(V_{BE}^{(n)}) \frac{V_{BE}^{(n)} - V_{BE}^{(n-1)}}{f(V_{BE}^{(n)}) - f(V_{BE}^{(n-1)})} \tag{5}$$

It converges super-linearly and is derivative-free.

## 4 Python Implementation

### 4.1 Code Description

The Python implementation defines the nonlinear function and its derivative, implements all three methods, tracks iteration history and absolute residual errors, and plots convergence curves, bar charts, and log-scale error plots. The code is modular, fully commented, and ensures reproducibility.

### 4.2 Complete Python Code

```
import math
import matplotlib.pyplot as plt

# GIVEN PARAMETERS
IS = 1e-12
IB = 4.545e-05
eta = 1
VT = 0.025

# DEFINE FUNCTIONS
def f(V):
    return IS*(math.exp(V/(eta*VT))-1)-IB
def df(V):
    return (IS/(eta*VT))*math.exp(V/(eta*VT))
```

```
# METHODS
def bisection(a,b,tol):
    history,error_history=[],[]
    iters=0
    while abs(b-a)>tol:
        c=(a+b)/2
        history.append(c)
        error_history.append(abs(f(c)))
        if f(a)*f(c)<0: b=c
        else: a=c
        iters+=1
    return c,iters,history,error_history

def newton(x0,tol):
    history,error_history=[],[]
    iters=0
    while True:
        x1=x0-f(x0)/df(x0)
        history.append(x1)
        error_history.append(abs(f(x1)))
        if abs(x1-x0)<tol: break
        x0=x1
        iters+=1
    return x1,iters,history,error_history

def secant(x0,x1,tol):
    history,error_history=[],[]
    iters=0
    while True:
        x2=x1-f(x1)*(x1-x0)/(f(x1)-f(x0))
        history.append(x2)
        error_history.append(abs(f(x2)))
        if abs(x2-x1)<tol: break
        x0,x1=x1,x2
        iters+=1
    return x2,iters,history,error_history

# RUN METHODS
tol=1e-12
b_V,b_it,b_hist,b_error=bisection(0.3,0.8,tol)
n_V,n_it,n_hist,n_error=newton(0.7,tol)
s_V,s_it,s_hist,s_error=secant(0.6,0.8,tol)

# PLOTS
colors=['skyblue','lightcoral','lightgreen']
methods=["Bisection","Newton-Raphson","Secant"]
iterations=[b_it,n_it,s_it]

plt.figure(figsize=(10,6))
plt.plot(b_hist,label="Bisection",marker='o',color=colors[0])
```

```
plt.plot(n_hist,label="Newton-Raphson",marker='s',color=colors
    [1])
plt.plot(s_hist,label="Secant",marker='^',color=colors[2])
plt.xlabel("Iteration")
plt.ylabel("V_BE (Volts)")
plt.title("Convergence Comparison")
plt.grid(True)
plt.legend()
plt.show()

plt.figure(figsize=(7,5))
plt.bar(methods,iterations,color=colors)
plt.ylabel("Iterations Required")
plt.title("Iteration Count Comparison")
plt.grid(axis='y')
plt.show()

plt.figure(figsize=(8,5))
bars=plt.bar(methods,iterations,color=colors)
for bar in bars:
    yval=bar.get_height()
    plt.text(bar.get_x()+bar.get_width()/2,yval+0.5,int(yval),
             ha='center',va='bottom',fontsize=10)
plt.xlabel("Method",fontsize=12)
plt.ylabel("Number of Iterations",fontsize=12)
plt.title("Iteration Comparison: Bisection vs Newton-Raphson vs
    Secant",fontsize=14)
plt.grid(axis='y',linestyle='--',alpha=0.6)
plt.ylim(0,max(iterations)+4)
plt.savefig("iteration_comparison_bar_chart.png")
plt.show()

plt.figure(figsize=(10,6))
plt.semilogy(range(1,len(b_error)+1),b_error,label="Bisection",
    marker='o',color=colors[0])
plt.semilogy(range(1,len(n_error)+1),n_error,label="Newton-
    Raphson",marker='s',color=colors[1])
plt.semilogy(range(1,len(s_error)+1),s_error,label="Secant",
    marker='^',color=colors[2])
plt.xlabel("Iteration Number")
plt.ylabel("Absolute Residual Error |f(V_BE)| (V)")
plt.title("Convergence Rate: Absolute Residual Error vs Iteration
    ")
plt.grid(True,which="both",ls="--",linewidth=0.5)
plt.legend()
plt.savefig("convergence_error_plot.png")
plt.show()
```

# 5 Convergence Testing and Performance Evaluation

## 5.1 Computed Results

Using a tolerance of $10^{-12}$, the results are summarized below:

| Method | $V_{BE}$ (V) | Iterations |
|---|---|---|
| Bisection | 0.458416 | 40 |
| Newton-Raphson | 0.458416 | 11 |
| Secant | 0.458416 | 13 |

Table 1: Convergence results of numerical methods

## 5.2 Convergence Curves

Figure 1 shows the convergence of each method toward $V_{BE}$. Newton-Raphson converges fastest, followed by Secant, while Bisection is slower but reliable.

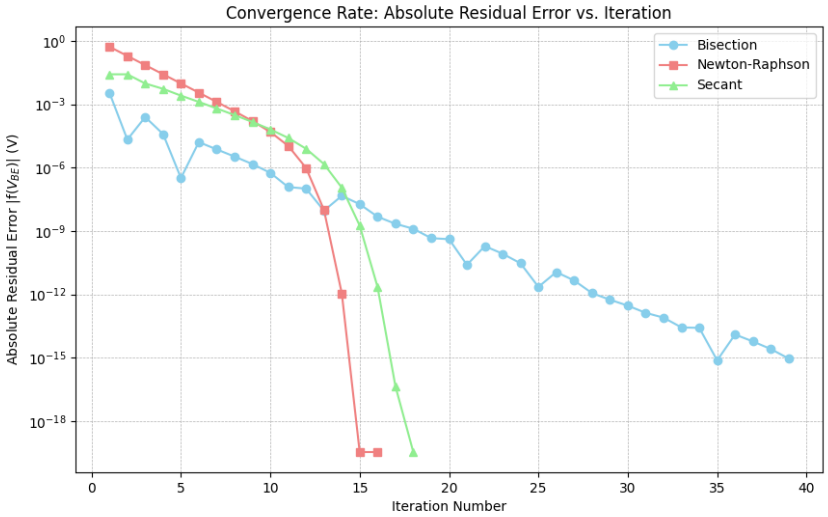



Figure 1: Absolute residual error vs iteration for Bisection, Newton-Raphson, and Secant methods.

## 5.3 Iteration Comparison

The iteration counts highlight computational efficiency. Newton-Raphson requires the fewest iterations, making it optimal for fast simulations, while Bisection provides guaranteed convergence.

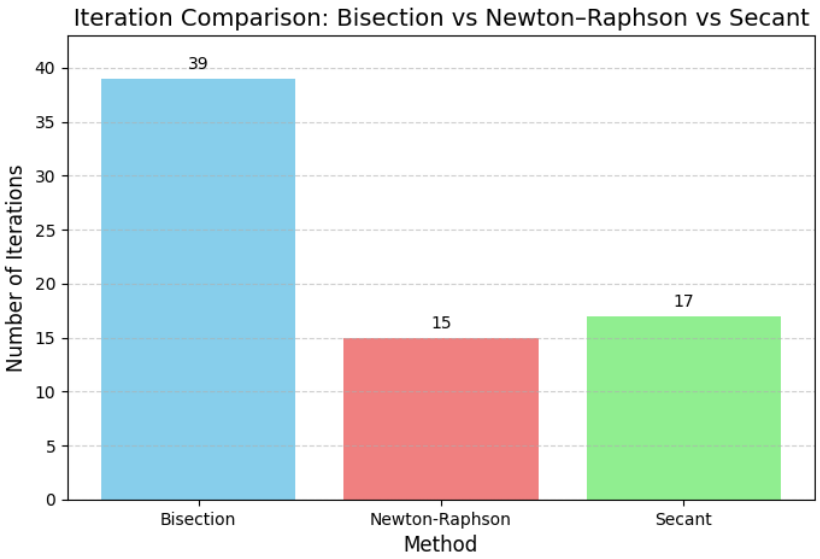



Figure 2: Iteration comparison among Bisection, Newton-Raphson, and Secant methods.

# 6 Optimization Insights and Real-World Relevance

The analysis provides practical insights:

- **Bisection:** Highly robust, useful as a fallback method.
- **Newton-Raphson:** Fastest convergence, preferred when derivatives are available.
- **Secant:** Efficient without derivatives, balances speed and simplicity.

In real-world circuit design, these methods replicate SPICE-like transistor biasing computations. Engineers use such iterative methods to determine operating points, optimize resistor values, and ensure circuit stability.

# 7 Discussion of Results

All methods converge to the same physical $V_{BE}$, confirming correctness. Error plots clearly illustrate different convergence rates. Newton-Raphson provides the fastest and most precise solution, whereas Bisection, despite being slower, offers guaranteed convergence. Secant achieves a good compromise, being faster than Bisection without requiring derivative calculations.

# 8 Conclusion

This activity successfully applied numerical root-finding methods to a nonlinear BJT bias circuit. The Python implementation is fully reproducible, tracks iteration history, and generates convergence plots. Newton-Raphson is the most efficient method, Secant offers derivative-free flexibility, and Bisection guarantees reliable convergence. These methods are directly relevant to engineering practices in analog circuit design, SPICE simulations,

and semiconductor modeling, demonstrating both theoretical understanding and practical application.